



Understanding Focus Apps Using AI

Final Project Report

Project ID: P000239DS

Course: Data Science and AI Postgraduate Project

Industry Supervisor: Jeremy Nagel, Focus Bear

Academic Supervisor: Han Phan

Student Team:

Sajana Methsara Kulatunge Hewa Walgamage- S4112953

Moncy Kunjumon- S4068103

Trayeash Balaji- S4082382

Sreeraj Raju - S4107873

Kelerra Sylvester- S4119360

Abstract

This project developed an automated app analysis system to support Focus Bear in understanding the competitive landscape of focus, productivity, habit-tracking, planner, and digital self-control applications. The system reduces the limitations of manual competitor analysis by collecting and cleaning app data, scraping reviews, classifying product features, analysing user sentiment, identifying ADHD-related behavioural patterns, and presenting insights through an interactive Streamlit dashboard.

The final system includes an improved scrape pipeline, cleaned competitor dataset, review scraping workflow, regex and LLM-based feature extraction, RoBERTa sentiment analysis, ADHD/ND review analysis, survivorship analysis, and dashboard visualisation. A key technical contribution was validating LLM-generated Digital Self-Control Tool (DSCT) feature labels against the original Lyngs coding data. After mismatch review and DSCT prompt refinement, the independent LLM match rate improved from 71.43% to 78.57%, with an intermediate result of 75.00%. Overall, the project demonstrates how data science and AI can support scalable, evidence-based competitor analysis and generate actionable product insights for Focus Bear.

Table of Contents

Abstract	1
1. Introduction and Client Background	4
2. Project Scope, Aims and Objectives	4
3. Methodology and System Development	5
3.1 Data Collection and Competitor Cleaning.....	6
3.2 Review Scraping and Review Preparation	6
3.3 Feature Classification	6
3.4 Feature Matrix Generation	7
3.5 Sentiment and ADHD/ND Analysis	7
3.6 Survivorship Analysis.....	7
3.7 Dashboard Development.....	8
4. Data Science Techniques and Justification.....	8
5. Project Deliverables and Key Findings.....	9
5.1 Main Deliverables	9
5.2 Key Findings	10
6. Testing, Evaluation and Quality Assurance	11
7. Teamwork and Reflection.....	11
8. Ethical, Professional and Practical Considerations	12
9. Limitations and Future Work	13
10. Conclusion	13
References	15
Appendix	16
A. Code and Project Management	16
A.1 Repository Access	16
A.2 Project Management Access	16
B. Roles and Responsibilities.....	17
B.1 Team Members.....	17
B.2 Detailed Roles and Responsibilities.....	17
C. Virtual Collaboration	20
C.1 Tools Used	20
C.2 Meeting and Collaboration Frequency	20

C.3 Time Spent Working Together and Individually	20
D. Individual Self-Reflection	21
D.1 Sajana Methsara Kulatunge Hewa Walgamage	21
D.2 Moncy Kunjumon	21
D.3 Trayeash Balaji	21
D.4 Sreeraj Raju	22
D.5 Kelerra Sylvester	22
E. Overall Team Reflection	23

1. Introduction and Client Background

Focus and productivity applications help users manage attention, reduce distractions, build habits, organise tasks, and maintain routines. These applications are used by students, professionals, and users with attention-related difficulties, including ADHD. As the number of focus and productivity apps grows across Google Play, Apple App Store, Chrome Web Store, and web-based platforms, it becomes difficult for companies to manually analyse competitors and identify product gaps.

Focus Bear, the industry partner, develops productivity and habit-building software to support focus, structured routines, and digital wellbeing. A key challenge for Focus Bear is understanding how competing apps position themselves, what features they offer, and how users respond to those features. Competitor app descriptions, website text, and user reviews are mostly unstructured, and similar features are often described using different wording. For example, “website blocking”, “distraction prevention”, and “focus mode” may describe related functionality, but simple keyword matching may not interpret these differences accurately.

The project therefore aimed to build an automated app analysis system that could collect and clean competitor app data, scrape and analyse reviews, classify features, detect ADHD-related behavioural patterns, compare app survival patterns, and present useful insights through a dashboard.

2. Project Scope, Aims and Objectives

The original aim was to develop an AI-driven system to analyse focus applications and generate insights into their features, user sentiment, and overall effectiveness. The final project scope included six main components:

1. Competitor app discovery, scraping, and cleaning.
2. Data preparation and transformation into structured CSV outputs.
3. Review scraping and review-level sentiment/ADHD labelling.
4. Feature classification using regex-based feature flags and LLM-based extraction.
5. Review analysis, including sentiment, ADHD/ND behavioural analysis, and survivorship analysis.
6. Streamlit dashboard development for competitor, review, feature, ADHD, and survivorship insights.

The scope changed from the early proposal in two important ways. First, the competitor dataset was improved because the original scraped list included many irrelevant apps such as general AI chatbots, browsers, VPNs, games, wallpapers, office tools, and unrelated utilities. The pipeline was updated so that irrelevant apps were removed during ingestion and transformation rather than only hidden later in the dashboard.

Second, feature classification was refined based on supervisor feedback. Instead of forcing all apps into the Lyngs DSCT taxonomy, the final system separated apps into DSCT apps, habit tracker apps, and planner/productivity apps. DSCT apps used the Lyngs feature categories, while habit and planner apps used separate project-specific prompts. This improved conceptual accuracy because habit trackers and planners do not always operate as digital self-control tools.

The project achieved the following objectives:

- Expanded competitor discovery using verified seed apps and improved search keywords.
- Added pre-filtering, relevance scoring, and final cleaning rules.
- Produced a cleaned competitor dataset for dashboard use.
- Scraped metadata and reviews from major app platforms.
- Applied RoBERTa-based sentiment analysis.
- Detected ADHD-related reviews and behavioural themes.
- Built regex and LLM-based feature extraction pipelines.
- Validated LLM feature labels against Lyngs coding data.
- Improved independent LLM match rate from 71.43% to 78.57%.
- Added survivorship analysis using historical review snapshots.
- Improved dashboard usability, competitor analysis, and app drill-down functionality.

3. Methodology and System Development

The project followed a modular pipeline-based methodology. This allowed different components to be developed, tested, and integrated into a full competitive intelligence system.

3.1 Data Collection and Competitor Cleaning

The system collected app metadata, descriptions, website text, ratings, installs, pricing where available, and user reviews. The main data files included `apps_clean.csv`, `apps_all_clean.csv`, `websites.csv`, `reviews.csv`, `reviews_with_sentiment.csv`, and `app_metrics.csv`.

A major improvement was made to the scrape and cleaning pipeline. The updated flow was:

```
scrapers/seeds.yml → scrapers/discovery.yml → scrapers/scrape_pipeline.py → full_dump.jsonl  
→ normalize_apps.py → apps_all.csv → clean_apps.py → apps_clean.csv → Streamlit  
dashboard
```

The seed list was expanded using verified package IDs, App Store IDs, and Chrome Web Store IDs. The discovery rules were improved with search phrases related to focus, deep work, Pomodoro, website blocking, app blocking, screen-time control, digital wellbeing, ADHD planners, executive-function tools, habit tracking, routine planning, and study support. Stronger exclusion rules were also added for unrelated apps such as browsers, games, VPNs, office tools, AI assistants, finance tools, and unrelated media utilities.

A pre-filtering stage was added after scraping and before normalisation. In one run, the pre-filter kept 619 of 1514 scraped rows, showing that many irrelevant apps were removed before entering the curated dataset. Relevance scoring was also strengthened so that apps with focus, ADHD, Pomodoro, screen-time, blocker, habit, routine, planner, productivity, or time-management signals were prioritised. The final cleaning step applied excluded titles, excluded developers, excluded categories, weak-scope rules, high-relevance overrides, Chrome extension overrides, and manually verified force-include lists.

3.2 Review Scraping and Review Preparation

The review pipeline collected user reviews from selected competitor apps, mainly from Google Play and Apple App Store. It supported configuration by maximum reviews per app, store, country, language, review age, and overwrite options. To improve ADHD/ND coverage, the pipeline could scale review collection by increasing the review cap, widening the date range, and including English-speaking countries such as Australia, United States, United Kingdom, Canada, New Zealand, and Ireland.

A sweet-spot profiling process was used to test review caps such as 200, 500, 1000, and 2000 reviews per app. This helped balance review coverage, processing time, and computational cost.

3.3 Feature Classification

The system used two feature extraction approaches: regex-based feature flags and LLM-based classification.

Regex-based feature flags provided a fast and transparent method for detecting common feature signals such as blocking, timers, Pomodoro, ADHD support, gamification, scheduling, and related

productivity features. Regex rules were useful because they were auditable, but they could not fully understand context.

LLM-based classification was used for richer semantic interpretation. The LLM pipeline used separate prompt builders for DSCT apps, habit apps, and planner apps. The DSCT taxonomy followed Lyngs et al.'s four main feature clusters: block/removal, self-tracking, goal-advancement, and reward/punishment. Project-specific extensions such as custom rules, override friction, emergency access, and scheduling were kept separately.

The LLM workflow loaded app descriptions, website text, and reviews; routed apps into relevant app types; selected the correct prompt; returned structured JSON; converted the output into feature rows; and generated features_llm.csv.

3.4 Feature Matrix Generation

The feature matrix generation process converted feature outputs into dashboard-ready files such as features_long.csv, features_summary.csv, features_app_type_summary.csv, features_matrix_flags.csv, features_matrix_confidence.csv, features_matrix_review_hits.csv, and features_bundle.csv. These outputs allowed the dashboard to show feature counts, feature matrices, confidence values, evidence fields, and app-level feature summaries.

3.5 Sentiment and ADHD/ND Analysis

The original sentiment method used VADER, but this was replaced with a transformer-based RoBERTa model because app reviews are often short, informal, and context-dependent. The improved pipeline generated sentiment-labelled review files such as reviews_with_ml_sentiment.csv and reviews_with_sentiment.csv.

The ADHD review analysis identified 252 ADHD-related reviews: 180 positive, 24 neutral, and 48 negative. Behavioural theme detection found that many ADHD-related reviews focused on attention support, task management, concentration, attention regulation, task completion, and productivity habits.

To extend beyond keyword matching, LLM-based behavioural classification was implemented using zero-shot classification with DistilBART MNLI. This identified 72 ADHD-related behavioural reviews across four categories: Time Management, Motivation and Reward, Task Management, and Distraction Management.

3.6 Survivorship Analysis

The project also included a Survivorship Analysis page using 2019 review snapshots from Google Play, Apple App Store, and Chrome Web Store. Apps were mapped to the Lyngs self-regulation app registry and manually verified as Active, Stale, or Discontinued as of May 2026.

The survivorship dashboard included an app status table, feature mention rates by survival status, rating trends over time, individual review exploration, and methodology notes. This gave Focus Bear a historical perspective on which features may be associated with longer-term app survival.

3.7 Dashboard Development

The Streamlit dashboard presented the project outputs through pages such as Overview, Competitors, Sentiment Analysis, Feature Matrix, ADHD Analysis, Survivorship Analysis, and Summary. The dashboard was updated to read from `apps_clean.csv`, ensuring that only cleaned competitor data was displayed.

Dashboard improvements included total app counts, average ratings, total ADHD reviews, top-rated competitor apps, platform split analysis, and individual app drill-down. A minimum threshold of 50 ratings was added to make competitor rankings more reliable. Missing ratings and installs were formatted as “No rating” and “Not available” instead of raw missing values. The Feature Matrix page was simplified to focus on clear outputs such as top detected features, feature distribution, app area summaries, and insight summaries.

4. Data Science Techniques and Justification

NLP was central to the project because app descriptions, website text, and reviews contain the main evidence about features and user experience. NLP allowed the system to convert unstructured text into structured outputs such as sentiment labels, feature labels, behavioural themes, and app-level metrics.

Rule-based cleaning and relevance scoring were used because broad app scraping can collect irrelevant results. These rules improved data quality before downstream AI analysis. Regex-based feature detection was used as a fast, explainable baseline for detecting common app features. However, because regex cannot fully understand context, LLM-based classification was added for deeper semantic interpretation.

LLM-based feature classification was justified because competitor apps describe similar functionality in varied ways. The LLM was especially useful for interpreting app descriptions and reviews, but validation showed that prompt design was critical. The DSCT prompt had to be refined so that general focus timers were not automatically classified as goal-advancement, reward systems were not treated as blocking unless access was actually blocked, and general productivity wording was not counted unless clear evidence existed.

Taxonomy-based classification improved consistency. DSCT apps used Lyngs et al.’s categories, while habit and planner apps used separate project-specific categories. This avoided forcing all apps into an unsuitable taxonomy.

RoBERTa-based sentiment analysis was used because transformer models are better suited than VADER for modern app-review language. Manual validation was used to evaluate sentiment predictions, feature classification quality, dataset relevance, and app survival status. Rule-based

and LLM-based ADHD detection were combined because some reviews explicitly mention ADHD, while others describe ADHD-related behaviour without using the term.

Survivorship analysis added a historical and strategic layer by comparing historical review patterns with current app survival status. This helped move the project beyond descriptive analytics toward decision support.

5. Project Deliverables and Key Findings

5.1 Main Deliverables

The project produced the following main deliverables:

Deliverable	Description
Cleaned competitor dataset	apps_clean.csv containing relevant focus, ADHD, timer, habit, planner, blocker, study-support, and wellbeing apps
Review dataset	reviews.csv for sentiment, ADHD/ND, and behavioural analysis
Sentiment outputs	RoBERTa-labelled review files such as reviews_with_ml_sentiment.csv
App metrics	Aggregated review-level insights in app_metrics.csv
Feature outputs	Regex and LLM feature classification outputs
Feature matrices	Dashboard-ready feature matrix, confidence, evidence, and bundle files
Validation outputs	Lyngs validation results, mismatch files, and by-feature summaries
ADHD outputs	ADHD review, theme, and behavioural analysis outputs
Survivorship analysis	Historical review and app survival analysis page
Dashboard	Streamlit dashboard with competitor, sentiment, feature, ADHD, survivorship, and summary views

5.2 Key Findings

The data cleaning process showed that competitor discovery requires both broad search coverage and strong filtering. Expanding seed apps and search phrases improved discovery, while exclusions, relevance scoring, and manual review prevented unrelated apps from entering the final dataset.

The LLM feature classification process showed clear improvement through validation and prompt refinement. The initial match rate against the original Lyngs coding was 71.43%. After stricter DSCT prompt rules, the match rate improved to 75.00%, reducing mismatches from 16 to 14. Further refinement improved the independent LLM match rate to 78.57%. The overall validation progression was:

71.43% → 75.00% → 78.57%

This result shows that LLMs can support feature classification, but they require structured taxonomy rules, mismatch review, and prompt calibration. It also suggests that an independent LLM may not reach full agreement with original human coding without calibration or human review.

The project also found that app type separation is important. Habit trackers and planners may support productivity and self-regulation, but they do not always fit the DSCT framework. Separating DSCT, habit, and planner apps improved the conceptual accuracy of the analysis.

The sentiment and ADHD analysis found 252 ADHD-related reviews. Most were positive, with 180 positive, 24 neutral, and 48 negative reviews. ADHD-related reviews often focused on attention support, concentration, task management, routine, motivation, and productivity habits. The LLM behavioural analysis identified 72 ADHD-related behavioural reviews, with Time Management and Motivation/Reward being the most common categories.

The survivorship analysis found that active apps showed higher mention rates for reward-related and ADHD-related features, while nudge-related features were more common in discontinued apps. These findings should be interpreted cautiously, but they provide useful historical insight into app survival patterns.

The dashboard improved stakeholder usability by presenting cleaned competitor data, top competitor rankings, platform distribution, feature matrices, ADHD insights, survivorship results, and app-level drill-down views in one interface.

6. Testing, Evaluation and Quality Assurance

Testing and evaluation were applied across the dataset, models, validation scripts, and dashboard.

For data quality, the team repeatedly reviewed scraped, cleaned, and dropped app files. `apps_dropped.csv` was checked to recover relevant apps that were incorrectly removed. The pipeline combined automated rules with manual review, including excluded titles, excluded developers, excluded categories, weak-scope rules, high-relevance overrides, Chrome extension overrides, and force-include lists.

For review scraping, the team used configurable parameters and sweet-spot profiling to test review caps before scaling. This reduced unnecessary processing while supporting larger review collection for ADHD/ND analysis.

For LLM feature classification, the team created a ground-truth validation file from the original Lyngs coding data. The validation script compared original Lyngs labels with LLM-generated labels and produced validation results, mismatch files, summary files, and by-feature outputs. The validation progression from 71.43% to 75.00% to 78.57% demonstrates measurable quality improvement.

For sentiment analysis, manual validation was used to compare RoBERTa predictions with human judgement. For survivorship analysis, app survival status was manually verified as Active, Stale, or Discontinued because automated store data may not fully reflect app survival.

For dashboard quality assurance, stakeholder feedback was used to improve readability, visual separation, metric relevance, ranking reliability, and app drill-down. The dashboard was also checked to ensure it used the cleaned dataset and handled missing values properly.

The project also maintained a structured codebase with separate modules for scraping, cleaning, review collection, sentiment analysis, feature extraction, feature matrix generation, validation, survivorship analysis, and dashboard display.

7. Teamwork and Reflection

The team divided responsibilities across the major system components while working toward a shared project goal.

Sajana Methsara Kulatunge Hewa Walgamage – Competitor List, Scrape Pipeline and Data Cleaning

Sajana improved the scrape pipeline and competitor cleaning process. This included expanding verified seeds, improving discovery keywords, adding exclusion rules, introducing pre-filtering, strengthening relevance scoring, reviewing dropped apps, recovering relevant apps, adding high-relevance overrides, and updating the dashboard to read from `apps_clean.csv`.

Sreeraj Raju – Sentiment Analysis and ADHD Behavioural Analysis

Sreeraj improved the review analysis pipeline by replacing VADER with RoBERTa, developing ADHD review detection, identifying behavioural themes, adding automated ADHD insights, and implementing LLM-based behavioural classification using DistilBART MNLI.

Moncy Kunjumon – Dashboard UX and Functionality Enhancements

Moncy improved the dashboard's usability and analytical value. His work included dashboard statistics, top-rated competitor app analysis, platform split analysis, individual app drill-down, card-style layouts, visual hierarchy, and clearer separation between dashboard sections.

Kelerra Sylvester – AI Feature Classification and Validation

Kelerra contributed to the LLM-based feature classification pipeline. This included updating the taxonomy, creating separate prompt logic for DSCT apps, habit trackers, and planners, refining DSCT prompt rules, generating feature matrix outputs, and validating LLM results against the original Lyngs coding dataset. This work improved the independent LLM match rate from 71.43% to 78.57%.

Trayeash Balaji – Full Pipeline Integration, Review Pipeline and Survivorship Analysis

Trayeash supported the end-to-end pipeline, including app discovery, scraping, review collection, sentiment and ADHD labelling, review aggregation, regex feature flags, LLM feature extraction, feature matrix generation, dashboard workflow, survivorship analysis, setup documentation, and known issue fixes.

The project demonstrated strong teamwork because each member contributed to a different part of the pipeline, while the final system integrated these parts into one workflow. The team also responded to supervisor feedback by improving feature classification, separating app taxonomies, cleaning the competitor dataset, and validating results.

A key lesson was that AI systems require ongoing validation. LLM outputs improved after prompt refinement, but they still required ground-truth comparison and mismatch review. Another lesson was that data quality strongly affects downstream analysis; irrelevant apps in the dataset reduce the usefulness of feature and dashboard results. The team also learned that dashboards should prioritise clear stakeholder insights rather than excessive technical detail.

8. Ethical, Professional and Practical Considerations

The project used publicly available app information and reviews. No private or sensitive personal data was intentionally collected. However, reviews may include personal experiences, so the analysis focused on aggregate insights rather than identifying users.

User reviews may contain bias because people who leave reviews may have stronger positive or negative experiences than average users. ADHD-related reviews may also be under-detected if users describe attention difficulties without explicitly mentioning ADHD.

LLMs may misclassify features when evidence is ambiguous. The validation process showed that prompt refinement improved results, but remaining mismatches mean the outputs should be treated as decision-support evidence rather than perfect truth.

Competitor cleaning rules can influence which apps are included or excluded. To reduce unjustified filtering, the project used explicit rules, relevance scoring, force-include lists, and manual review. Survivorship analysis also involved manual judgement, so the results should be interpreted as exploratory rather than definitive.

9. Limitations and Future Work

The independent LLM match rate improved to 78.57%, but mismatches remain compared with the original Lyngs coding dataset. Future work should refine prompts further, expand validation data, and develop a calibration approach for known mismatch patterns.

Competitor discovery depends on search keywords, seed apps, platform metadata, and cleaning rules. Some relevant apps may still be missed if they use unexpected terminology. Future work should periodically review dropped apps, update exclusion and force-include lists, and add more verified seed apps.

Install counts were incomplete across some platforms, so some popularity estimates relied on rating counts. These estimates are useful for comparison but should not be treated as exact values.

ADHD-related review detection remains challenging because many relevant reviews may not explicitly mention ADHD. Future work could use larger labelled datasets, fine-tuned models, and better behavioural clustering.

Review-level drill-down could be improved by strengthening links between application records and review records. Survivorship analysis could also be improved with more historical snapshots and stronger app-status verification.

Future dashboard improvements could include an App Type / Focus Category filter, improved export functionality, better recommendation generation, and more advanced decision-support summaries for Focus Bear.

10. Conclusion

This project successfully developed an automated app analysis system for Focus Bear. The system supports competitor analysis by combining app discovery, competitor cleaning, review scraping, regex feature flags, LLM feature classification, taxonomy-based analysis, sentiment analysis, ADHD/ND behavioural analysis, survivorship analysis, validation, and dashboard visualisation.

The project achieved several important outcomes. The scrape pipeline was improved to discover more relevant competitors while removing irrelevant apps earlier in the pipeline. The review pipeline supported sentiment and ADHD/ND analysis at scale. The feature classification pipeline was aligned with Lyngs et al.'s DSCT taxonomy while using separate prompts for habit and planner apps. The LLM validation improved from 71.43% to 78.57%, showing measurable progress through prompt refinement and mismatch review. The sentiment pipeline improved by replacing VADER with RoBERTa, and ADHD analysis identified 252 ADHD-related reviews and important behavioural themes. Survivorship analysis added a historical perspective by comparing app survival status with feature and review patterns. The dashboard integrated these insights into an accessible tool for competitor analysis.

Overall, the project demonstrates how AI and data science can support scalable, evidence-based competitor analysis. While further work is needed in feature calibration, review linking, competitor discovery, and survivorship verification, the final system provides a strong foundation for Focus Bear to understand the focus app market and identify product improvement opportunities.

References

Aggarwal, C.C. (2018) *Machine Learning for Text*. Cham: Springer.

Bird, S., Klein, E. and Loper, E. (2009) *Natural Language Processing with Python*. Sebastopol: O'Reilly Media.

Jurafsky, D. and Martin, J.H. (2021) *Speech and Language Processing*. 3rd edn. Draft. Available at: <https://web.stanford.edu/~jurafsky/slp3/>

Liu, B. (2012) *Sentiment Analysis and Opinion Mining*. San Rafael: Morgan & Claypool.

Lyngs, U., Lukoff, K., Slovak, P., Seymour, W., Webb, H., Jirotko, M., Zhao, J., Van Kleek, M. and Shadbolt, N. (2019) 'Self-control in cyberspace: Applying dual systems theory to a review of digital self-control tools', *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*.

Manning, C.D., Raghavan, P. and Schütze, H. (2008) *Introduction to Information Retrieval*. Cambridge: Cambridge University Press.

RMIT University (2026) *Data Science and AI Postgraduate Project: Project Report Final Report Rubric*. RMIT University Canvas.

Appendix

A. Code and Project Management

A.1 Repository Access

The project code was developed and managed through a shared GitHub repository. The repository contains the source code, scraping pipeline, ETL scripts, review analysis scripts, LLM feature classification files, feature matrix generation scripts, survivorship analysis files, Streamlit dashboard code, configuration files, and supporting documentation.

Repository link: <https://github.com/Focus-Bear-research/automated-app-analysis-system>

The main code areas in the repository include:

- scrapers/ – app discovery, app scraping, review scraping, and store-specific scraping scripts.
- etl/ – data normalisation, cleaning, website scraping, app card building, review aggregation, and feature matrix generation.
- llm/ – sentiment pipeline, feature extraction, feature flags, and LLM-based feature classification.
- scripts/ – improved sentiment analysis, ADHD sentiment analysis, ADHD theme detection, and LLM ADHD classifier scripts.
- data/curated/ – cleaned datasets and generated analysis outputs.
- survivorship_analysis.py – survivorship analysis using historical review snapshots.
- app.py – Streamlit dashboard.
- README files – documentation of each development area and instructions for running the system.

A.2 Project Management Access

The team mainly used shared online tools for project management and communication.

Project management / collaboration tools used:

- **GitHub** – used for code storage, pipeline files, version control, and documentation.
- **Google Docs** – used for collaborative report writing and editing.
- **Google Drive** – used to share README files, screenshots, datasets, drafts, and supporting material.
- **Group chat** – used for daily communication, reminders, quick questions, file requests, and progress updates.
- **Online meetings / Google Meet** – used for group discussions and supervisor meetings.
- **Supervisor feedback** – used to guide project direction and improve technical components.

B. Roles and Responsibilities

B.1 Team Members

Team Member	Student ID	Main Responsibility
Sajana Methsara Kulatunge Hewa Walgamage	S4112953	Competitor list, scrape pipeline improvements, and data cleaning
Moncy Kunjumon	S4068103	Dashboard user experience and functionality enhancements
Trayeash Balaji	S4082382	Full pipeline integration, review pipeline, and survivorship analysis
Sreeraj Raju	S4107873	Sentiment analysis and ADHD behavioural analysis
Kelerra Sylvester	S4119360	LLM feature classification, taxonomy updates, and validation

B.2 Detailed Roles and Responsibilities

Sajana Methsara Kulatunge Hewa Walgamage – Competitor List, Scrape Pipeline and Data Cleaning

Sajana was responsible for improving the competitor app discovery and cleaning pipeline. His main deliverable for the industry partner was a cleaner and more reliable competitor app dataset that could be used confidently by the downstream analysis and dashboard components.

His work included:

- Improving the scrape pipeline so irrelevant apps were removed during ingestion and transformation.
- Expanding the seed list using verified Google Play package IDs, App Store IDs, and Chrome Web Store IDs.
- Improving discovery keywords for focus apps, ADHD apps, Pomodoro apps, habit trackers, planners, routine apps, screen-time tools, and digital wellbeing apps.
- Adding stronger exclusion rules for unrelated apps such as AI chatbots, browsers, VPNs, games, wallpapers, office tools, and unrelated utilities.
- Adding pre-filtering before normalisation.
- Improving relevance scoring in the normalisation stage.
- Updating final cleaning rules in `clean_apps.py`.
- Adding high-relevance overrides so small but relevant apps were not removed unfairly.
- Reviewing dropped apps and recovering relevant apps through force-include rules.
- Updating the dashboard to read from `apps_clean.csv`.

This work improved data quality before feature classification, sentiment analysis, ADHD analysis, and dashboard visualisation.

Moncy Kunjumon – Dashboard UX and Functionality Enhancements

Moncy was responsible for improving the usability, readability, and analytical value of the Streamlit dashboard. His main deliverable for the industry partner was a clearer dashboard that allowed Focus Bear to understand competitor apps and app-level insights more easily.

His work included:

- Improving dashboard statistics and summary metrics.
- Displaying total analysed apps, average rating, and total ADHD reviews.
- Adding a Top Rated Competitor Apps section.
- Adding a minimum threshold of 50 ratings to improve ranking reliability.
- Adding platform split analysis for Play Store, App Store, and Web/Chrome apps.
- Creating an individual app drill-down section.
- Displaying app rating, estimated installs, platform, developer information, genre, rating count, and detailed app data.
- Improving visual hierarchy, layout spacing, section boundaries, and dashboard readability.
- Redesigning sections using card-style layouts.
- Improving dashboard clarity based on stakeholder feedback.

This work made the dashboard more suitable for presentation and industry use.

Trayeash Balaji – Full Pipeline Integration, Review Pipeline and Survivorship Analysis

Trayeash was responsible for supporting and documenting the full automated app analysis pipeline. His main deliverable for the industry partner was a reproducible end-to-end workflow that connected scraping, cleaning, review analysis, feature extraction, dashboard outputs, and survivorship analysis.

His work included:

- Documenting the full project structure and setup process.
- Supporting app discovery and metadata scraping across Google Play, Apple App Store, and Chrome Web Store.
- Supporting review scraping with configurable parameters such as review limits, stores, countries, languages, and time range.
- Supporting sentiment and ADHD labelling workflow.
- Supporting app-level review insight aggregation.
- Documenting regex-based feature flag extraction.
- Documenting LLM-based feature extraction.
- Supporting feature matrix generation.
- Documenting dashboard launch workflow.
- Developing and documenting survivorship analysis using historical 2019 review snapshots.
- Mapping historical apps to app survival status such as Active, Stale, and Discontinued.

- Documenting known issues and fixes to support reproducibility.

This contribution helped connect the individual technical components into a full competitive intelligence pipeline.

Sreeraj Raju – Sentiment Analysis and ADHD Behavioural Analysis

Sreeraj was responsible for improving the review analysis pipeline. His main deliverable for the industry partner was a stronger review analysis component that could identify review sentiment and ADHD-related behavioural patterns.

His work included:

- Replacing the original VADER sentiment analysis approach with a RoBERTa-based model.
- Creating sentiment-labelled review outputs.
- Performing manual validation of sentiment predictions.
- Developing ADHD-related review detection.
- Generating ADHD sentiment statistics.
- Identifying ADHD behavioural themes such as attention support and task management.
- Creating ADHD behavioural insights and recommendations.
- Implementing LLM-based behavioural classification using DistilBART MNLI.
- Identifying behavioural categories such as Time Management, Motivation and Reward, Task Management, and Distraction Management.
- Integrating ADHD-related analysis outputs into the dashboard.

This contribution expanded the project beyond general sentiment analysis and provided deeper insight into ADHD-related user experiences.

Kelerra Sylvester – LLM Feature Classification, Taxonomy Updates and Validation

Kelerra was responsible for the LLM-based feature classification and validation component. Her main deliverable for the industry partner was a structured feature classification pipeline that could classify competitor app features using app descriptions, website text, and reviews.

Her work included:

- Updating the feature taxonomy.
- Separating apps into DSCT apps, habit tracker apps, and planner/productivity apps.
- Keeping the four Lyngs DSCT feature clusters: block/removal, self-tracking, goal-advancement, and reward/punishment.
- Creating separate prompt logic for DSCT, habit, and planner apps.
- Improving the DSCT prompt using supervisor feedback.
- Generating LLM feature classification outputs.
- Updating feature matrix generation outputs.
- Creating validation files against the original Lyngs coding data.
- Reviewing mismatch patterns.
- Refining the DSCT prompt with clearer rules around timers, blocking, self-tracking, goal-advancement, and reward/punishment.
- Improving the independent LLM match rate from 71.43% to 78.57%.

This work improved the reliability and interpretability of the feature classification component.

C. Virtual Collaboration

The team collaborated virtually throughout the project using online communication and file-sharing tools.

C.1 Tools Used

The team used the following tools:

- **Google Docs** for collaborative report writing.
- **Google Drive** for sharing README files, screenshots, datasets, drafts, and supporting material.
- **GitHub** for source code, project files, version control, and technical documentation.
- **Group chat** for quick communication, reminders, questions, and file sharing.
- **Google Meet / online meetings** for group discussions and supervisor meetings.
- **Email / Canvas communication** where required for academic and supervisor communication.

C.2 Meeting and Collaboration Frequency

The team communicated regularly through group chat and online updates. Formal and informal discussions were held throughout the semester, especially when preparing deliverables, responding to supervisor feedback, integrating components, and preparing the final report.

The team also participated in supervisor meetings and used feedback from the academic and industry supervisors to refine project direction. For example, supervisor feedback led to improvements in feature classification, including the decision to use Lyngs et al.'s DSCT taxonomy for DSCT apps while using separate prompts for habit trackers and planners.

C.3 Time Spent Working Together and Individually

Most technical development was completed individually according to assigned responsibilities. Each member worked independently on their own component, such as data cleaning, dashboard development, feature classification, review analysis, or pipeline integration. Group time was mainly used for coordination, progress updates, report preparation, file sharing, feedback review, and final integration.

Approximate working pattern:

- **Individual work:** Most of the coding, testing, README writing, analysis, and debugging were completed individually by each member.
- **Virtual group work:** The team spent time together online discussing progress, clarifying tasks, reviewing requirements, preparing the report, and integrating contribution summaries.
- **Collaborative writing:** The final report was prepared through Google Docs, where members could review, modify, and add their sections.

This structure allowed members to work efficiently on specialised components while still contributing to one final integrated project.

D. Individual Self-Reflection

D.1 Sajana Methsara Kulatunge Hewa Walgamage

I learned that data quality is one of the most important parts of a data science system. At the beginning, the competitor app list included many irrelevant apps such as browsers, games, VPN tools, office apps, AI chatbots, wallpapers, and unrelated utilities. This showed me that scraping more data does not always mean getting better data. If the dataset is noisy, the downstream analysis becomes less reliable.

What worked well was improving the scrape pipeline so that irrelevant apps were removed earlier during ingestion and transformation. Expanding the seed list helped discover more relevant competitors, while exclusion rules, relevance scoring, and force-include rules improved the quality of the final dataset. Reviewing `apps_dropped.csv` also helped identify relevant apps that were incorrectly removed.

One thing that did not work well at first was relying on broad discovery rules. Some generic keywords such as “timer”, “planner”, or “habit” brought in apps that were not true competitors. This taught me that broad keywords must be balanced with strong scope rules and manual review. Next time, I would try to introduce app-type classification earlier in the pipeline so that focus apps, ADHD support apps, planners, habit trackers, and digital wellbeing tools can be grouped more accurately.

D.2 Moncy Kunjumon

I learned that dashboard design is not only about showing charts but also about making the analysis easy for stakeholders to understand. A dashboard can contain useful results, but if the layout is too technical or unclear, users may not be able to interpret the findings properly.

What worked well was improving the dashboard structure, adding clearer metrics, creating a top-rated competitor section, improving platform analysis, and developing app-level drill-down. The 50-rating threshold also improved the reliability of the top-rated competitor list because it prevented apps with very few ratings from appearing as top competitors.

One thing that did not work well initially was that some dashboard sections were too technical or visually unclear. Some missing values also appeared as raw values such as `nan` or `0.0`, which looked unprofessional. Fixing these display issues taught me that data presentation is an important part of data science communication. Next time, I would improve the dashboard further by adding better filters, export options, and project-specific app categories.

D.3 Trayeash Balaji

Through this project, I learned the importance of building a reproducible end-to-end data science pipeline. The project had many connected stages, including app discovery, scraping, cleaning, review collection, sentiment analysis, feature extraction, feature matrix generation, dashboard display, and survivorship analysis. If one step is not clearly documented or connected, it becomes difficult for others to rerun or extend the system.

What worked well was documenting the project structure, setup process, pipeline commands, key data files, and dashboard workflow. This helped explain how the system works from data collection to final visualisation. Survivorship analysis also added a useful historical perspective by comparing older review snapshots with current app survival status.

One challenge was that multi-stage pipelines can be difficult to maintain because small issues in file paths, command arguments, dependencies, or missing data files can stop later stages from running. Some commands and known issues had to be documented carefully so that future users could avoid the same problems. Next time, I would try to add more automated tests, clearer error messages, and simpler setup scripts to make the pipeline easier to run.

D.4 Sreeraj Raju

Through this project, I learned that choosing the right model for the data context is very important. The original sentiment analysis approach used VADER, but app reviews are often short, informal, and context-dependent. VADER did not always classify these reviews reliably. Replacing it with a RoBERTa-based model improved the sentiment analysis pipeline.

What worked well was combining sentiment analysis with ADHD-related review detection and behavioural theme analysis. This made the review analysis more meaningful because it went beyond positive and negative sentiment and identified themes such as attention support, task management, time management, motivation, and distraction management.

One thing that did not work well was relying only on keyword-based ADHD detection. Some users describe ADHD-related struggles without directly using the word “ADHD”. This made it difficult to identify all relevant reviews using simple rules. The LLM-based behavioural classification helped address this limitation by detecting behavioural patterns based on meaning. Next time, I would improve this further by using a larger labelled dataset and testing fine-tuned models for ADHD-related behaviour detection.

D.5 Kelerra Sylvester

I learned that LLMs can be useful for feature classification, but they need careful prompt design, validation, and human review. At first, the independent LLM classification achieved a 71.43% match rate against the original Lyngs coding data. After reviewing mismatches, I found that the model sometimes over-detected DSCT categories. For example, it sometimes treated general focus timers as goal-advancement or reward-based apps as block/removal even when there was no actual blocking.

What worked well was refining the DSCT prompt with clearer rules around timers, blocking, self-tracking, goal-advancement, and reward/punishment. After improving the prompt, the match rate improved from 71.43% to 75.00%, and later to 78.57%. This showed that prompt refinement and mismatch review can improve LLM classification quality.

One thing that did not work well was expecting the LLM to independently reproduce the original Lyngs coding decisions perfectly. Some categories depend on subtle interpretation, and the LLM may not always apply the same reasoning as human coders. This taught me that LLM outputs should not be treated as ground truth without validation. Next time, I would use a calibration approach by reviewing known mismatch patterns and combining LLM outputs with human-reviewed rules.

E. Overall Team Reflection

Overall, the project showed that real-world data science work is iterative. The final solution was not produced in one attempt. Several early approaches had limitations, but those limitations helped improve the system.

The broad scrape pipeline initially collected irrelevant apps, so the team improved seed selection, discovery rules, exclusion rules, pre-filtering, relevance scoring, and manual review. The original sentiment analysis approach used VADER, but it was replaced with RoBERTa because VADER was not ideal for app-review language. The initial LLM feature classification achieved only 71.43% match rate, so the DSCT prompt was refined and validated again. The dashboard also needed improvements because some sections were too technical or unclear for stakeholders.

The main lesson was that data science projects require both technical development and continuous evaluation. Good results depend on clean data, suitable models, clear validation, and effective communication. The team also learned that failure and revision are part of the process. Each issue revealed something useful: noisy scraping showed the need for stronger data cleaning, VADER's limitations showed the need for a stronger sentiment model, and LLM mismatches showed the need for prompt refinement and calibration.

In future projects, the team would aim to integrate components earlier, validate outputs sooner, document decisions more consistently, and create clearer links between data files, code outputs, and dashboard visualisations.